SCHOOL OF APPLIED SCIENCE

A Mini Project Report

Of

Course Title: Python Programming

Course Code: 25MCAGL101

On

"Hospital Appointment Booking System"

Submitted partial fulfillment of requirement for the I semester

Master of Computer Applications (MCA)

Submitted By

RAKSHITH GOWDA
25SUPMCAGL102

Under the Guidance of

Ms. BHOOMIKA S

Assistant Professor

School of Applied Science, SNPSU

SAPTHAGIRI NPS UNIVERSITY

#14/5, Chikkasandra, Hesarghatta Main Road, Bengaluru – 560057

SAPTHAGIRI NPS UNIVERSITY

UNMATCHED EXCELLENCE, UNLIMITED POTENTIAL

SCHOOL OF APPLIED SCIENCE

CERTIFICATE

This is Certified that the Mini Project work entitled "**Hospital Appointment Booking System"** is a bonafide work carried out by **RAKSHITH GOWDA** bearing the university seat number **25SUPMCAGL102** in partial fulfillment for the award of Master of Computer Applications of the Sapthagiri NPS University Bengaluru during the year 2025-26. It is certified that all corrections/suggestions indicated for the internal assessment have been incorporated in the Mini Project report. The Mini Project report has been approved as it satisfies the academic requirements in respect of Mini Project work prescribed for the I Semester, Master of Computer Applications Degree**.**

Signature of Guide
Ms.Bhoomika S
Assistant Professor
School of Applied Science, SNPSU

Signature of Director
Dr. Jagannatha S
Director
School of Applied Science, SNPSU

Name of Examiners

1. _____

2. _____

Signature of Examiners

1. _____

2. _____

SAPTHAGIRI NPS UNIVERSITY
UNMATCHED EXCELLENCE, UNLIMITED POTENTIAL

SCHOOL OF APPLIED SCIENCE

DECLARATION

I, **RAKSHITH GOWDA** (**25SUPMCAGL102**), a student of First Semester Master of Computer Applications (MCA), hereby solemnly declare that the mini project report titled **Hospital Appointment Booking System** is a bonafide record of the original work carried out by me at the School of Applied Science, Sapthagiri NPS University, Bengaluru, during the academic year 2025 - 2026.

I further declare that the aforesaid work has been completed under the guidance and supervision of **Ms.Bhoomika S**, Assistant Professor, School of Applied Science. I hereby affirm that this project report has not been submitted, either wholly or in part, to this University or to any other institution for the award of any degree, diploma, or any other academic qualification.

Place: Bengaluru Rakshith Gowda

Date: 25SUPMCAGL102

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of a task would be incomplete without the mention of the people who made it possible and without any guidance and encouragement success would not have been possible.

I am grateful to the Sapthagiri NPS University and management for its ideas and inspiration for having provided us with the good infrastructure, laboratory, facilities, and inspiring staff which has made this Mini Project work successfully.

I would like to express my sincere gratitude to **Dr. Jagannatha S**, Director of the School of Applied Science, for all the facilities that have been extended to them throughout my work.

I would like to express my sincere gratitude to the Guide **Ms.Bhoomika S,** Assistant Professor, School of Applied Science for her valuable guidance and support.

Rakshith Gowda
25SUPMCAGL102

TABLE OF CONTENTS

Sl. No.	Contents	Page No.
1	Introduction	1-2
2	Requirement specifications	2-3
3	Design	4-5
4	Implementation	6-13
5	Conclusion	13-14
6	Bibliography	14-15

1. INTRODUCTION

Healthcare services are an essential part of human life. In modern society, hospitals receive a large number of patients every day, making appointment management a challenging task.

Traditionally, hospital appointments were managed manually through registers or by visiting the hospital reception. This manual process consumes time, creates long queues, and increases the chances of human errors.

With the advancement of technology, many hospitals are shifting towards digital systems to manage their services efficiently. A Hospital Appointment Booking System is an online application that allows patients to schedule appointments with doctors through the internet. This system reduces the time required for booking appointments and improves hospital management.

The Hospital Appointment Booking System developed in this project is implemented using Python programming language and MongoDB database. Python is widely used for web application development due to its simplicity and powerful frameworks. MongoDB is a NoSQL database that stores data in a flexible document format, making it suitable for modern web applications.

The system provides two main modules: User Module and Admin Module. The User module allows patients to register, login, and book appointments with doctors. The Admin module enables administrators to view all appointments, edit appointment details, and manage the system effectively.

The main objective of this project is to simplify the appointment booking process for patients and reduce the workload of hospital staff. The system stores all appointment information securely in the MongoDB database. This ensures data reliability and easy access.

The proposed system improves hospital efficiency, reduces waiting time for patients, and ensures better communication between doctors and patients.

2. REQUIREMENT SPECIFICATION

Requirement specification describes the hardware, software, and functional requirements needed to develop and run the system.

2.1 Hardware Requirements

The following hardware components are required to run the Hospital Appointment Booking System:

Processor: Intel Core i3 or higher

RAM: Minimum 4 GB

Hard Disk: Minimum 500 GB

Keyboard and Mouse

Monitor

These hardware components are sufficient for developing and testing the application.

2.2 Software Requirements

The following software tools are used to develop the system:

Operating System: Windows 10 / Windows 11 / Linux

Programming Language: Python

Database: MongoDB

Web Framework: Flask (or Django if used)

Code Editor: Visual Studio Code / PyCharm

Web Browser: Google Chrome / Firefox

Python is used as the main programming language because it provides easy syntax and powerful libraries. MongoDB is used for storing data because it is flexible and scalable.

2.3 Functional Requirements

Functional requirements describe what the system should do.

User Module

User Registration

Users can create an account by entering their personal details such as name, email, phone number, and password.

User Login

Registered users can log in using their email and password.

Book Appointment

Users can select a doctor, choose a date, and book an appointment.

View Appointment

Users can view the details of their booked appointments.

Logout

Users can log out of the system securely.

Admin Module

Admin Login

The administrator logs into the system using admin credentials.

View Appointments

Admin can view all booked appointments stored in the database.

Edit Appointment

Admin can modify appointment details such as date, time, or doctor.

Manage Users

Admin can view registered users.

Delete Appointment

Admin can cancel or remove appointments if necessary.

2.4 Non-Functional Requirements

Performance

The system should process requests quickly and respond efficiently.

Security

User data and appointment details should be securely stored.

Usability

The system interface should be simple and easy to use.

Reliability

The system should work without errors and maintain accurate data.

3. SYSTEM DESIGN

3.1 System Architecture

The Hospital Appointment Booking System follows a three-layer architecture:

Presentation Layer

This layer includes the user interface where users interact with the system through web pages.

Application Layer

This layer contains the Python application logic that processes user requests.

Database Layer

This layer stores data in MongoDB collections such as users, appointments, and admin details.

3.2 Database Design

MongoDB stores data in collections and documents.

Users Collection

This collection stores user details such as:

User ID

Name

Email

Password

Phone Number

Appointments Collection

This collection stores appointment details such as:

Appointment ID

User ID

Doctor Name

Appointment Date

Appointment Time

Status

Admin Collection

Stores administrator login credentials.

3.3 System Modules

The system is divided into the following modules:

User Registration Module

User Login Module

Appointment Booking Module

Admin Management Module

Appointment Editing Module

Each module performs a specific function within the system.

3.4 Data Flow

The data flow in the system follows these steps:

User registers and login information is stored in MongoDB.

User logs in to the system.

User books an appointment.

Appointment data is stored in the database.

Admin logs in to the admin panel.

Admin views or edits appointment details.

4.IMPLEMENTATION

Implementation describes how the system is developed using Python and MongoDB. The Hospital Appointment Booking System is implemented using Python with Flask framework. Flask is a lightweight web framework that simplifies web development. MongoDB is used as the backend database to store user information and appointment records.

4.1User Registration Implementation

Users enter their details in the registration form. The system validates the input and stores the data in the MongoDB users collection.

4.2User Login Implementation

The login module verifies the user credentials by comparing the entered data with the stored database records. If the credentials are correct, the user is redirected to the dashboard.

4.3Appointment Booking Implementation

Users can book appointments by selecting doctor name, date, and time. The system stores the appointment details in the MongoDB appointments collection.

4.4Admin Panel Implementation

The administrator logs into the admin dashboard and can perform the following actions:

View all appointments

Edit appointment details

Delete or cancel appointments

Manage user records

The admin interface helps in controlling the entire system.

4.5 Database Connectivity

Python connects to MongoDB using the PyMongo library. PyMongo allows Python applications to interact with MongoDB databases efficiently.

Example functions include:

Insert appointment data

Retrieve appointment records

Update appointment details

Delete appointment records

4.6 coding

```
from flask import Flask
from pymongo import MongoClient

app = Flask(__name__)

# MongoDB connection
client = MongoClient("mongodb://localhost:27017/") db
= client["hospital_db"]

users_collection = db["users"]
appointments_collection = db["appointments"]
admin_collection = db["admin"]

from flask import request, redirect, render_template

@app.route('/register', methods=['GET','POST']) def
register():
    if request.method == 'POST':
```

```
        name = request.form['name']
        email = request.form['email']
        password = request.form['password']

        user = {
            "name": name, "email":
            email, "password":
            password
        }

        users_collection.insert_one(user) return

        redirect('/login')

    return render_template("register.html")


@app.route('/login', methods=['GET','POST']) def
login():
    if request.method == 'POST': email
        = request.form['email']
        password = request.form['password']

        user = users_collection.find_one({
            "email": email,
            "password": password
        })

        if user:
            return redirect('/dashboard')
        else:
            return "Invalid Login"

    return render_template("login.html")
```

```
@app.route('/book', methods=['GET','POST']) def
book_appointment():
    if request.method == 'POST': patient_name
        = request.form['name'] doctor =
        request.form['doctor']
        date = request.form['date'] time
        = request.form['time']

        appointment = {
            "patient_name": patient_name,
            "doctor": doctor,
            "date": date, "time":
            time
        }

        appointments_collection.insert_one(appointment) return

        "Appointment Booked Successfully"

    return render_template("book.html")


@app.route('/admin_login', methods=['GET','POST']) def
admin_login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']

        admin = admin_collection.find_one({
            "username": username, "password":
            password
        })
```

```
        if admin:
            return redirect('/admin_dashboard') else:
            return "Invalid Admin Login"

    return render_template("admin_login.html")



@app.route('/admin_dashboard') def
admin_dashboard():
    appointments = appointments_collection.find()

    return render_template("admin_dashboard.html", appointments=appointments)


from bson.objectid import ObjectId

@app.route('/edit/<id>', methods=['GET','POST']) def
edit_appointment(id):

    if request.method == 'POST':
        doctor = request.form['doctor']
        date = request.form['date'] time =
        request.form['time']

        appointments_collection.update_one(
            {"_id": ObjectId(id)},
            {"$set": {  "doctor":
                doctor, "date":
                date, "time": time
            }}
        )
```

```
return redirect('/admin_dashboard')

appointment = appointments_collection.find_one({"_id": ObjectId(id)}) return
render_template("edit.html", appointment=appointment)
```

FIG:1

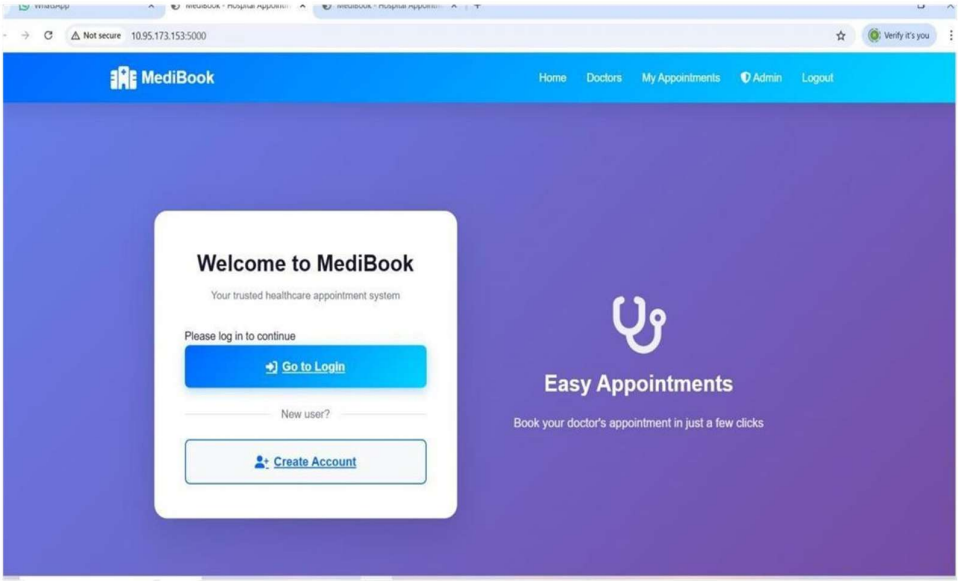


FIG2

-

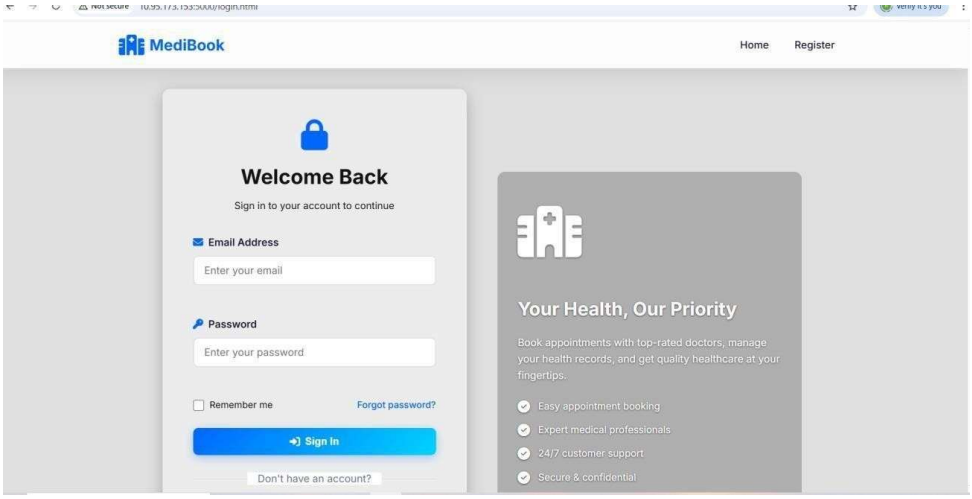

MediBook
Home
Register
Welcome Back
Sign in to your account to continue
Email Address
Enter your email
Password
Enter your password
Remember me
Forgot password?
Sign In
Don't have an account?
Your Health, Our Priority
Book appointments with top-rated doctors, manage your health records, and get quality healthcare at your fingertips.
Easy appointment booking
Expert medical professionals
24/7 customer support
Secure & confidential

FIG3

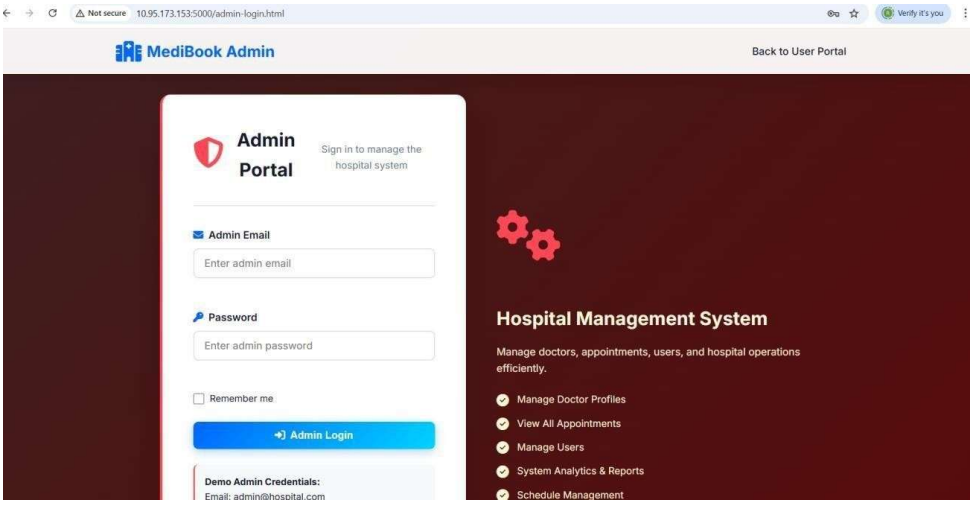

Not secure
10.95.173.153:5000/admin-login.html
Verify it's you
MediBook Admin
Back to User Portal
Admin Portal
Sign in to manage the hospital system
Admin Email
Enter admin email
Password
Enter admin password
Remember me
Admin Login
Demo Admin Credentials:
Email: admin@hospital.com
Hospital Management System
Manage doctors, appointments, users, and hospital operations efficiently.
Manage Doctor Profiles
View All Appointments
Manage Users
System Analytics & Reports
Schedule Management

FIG4

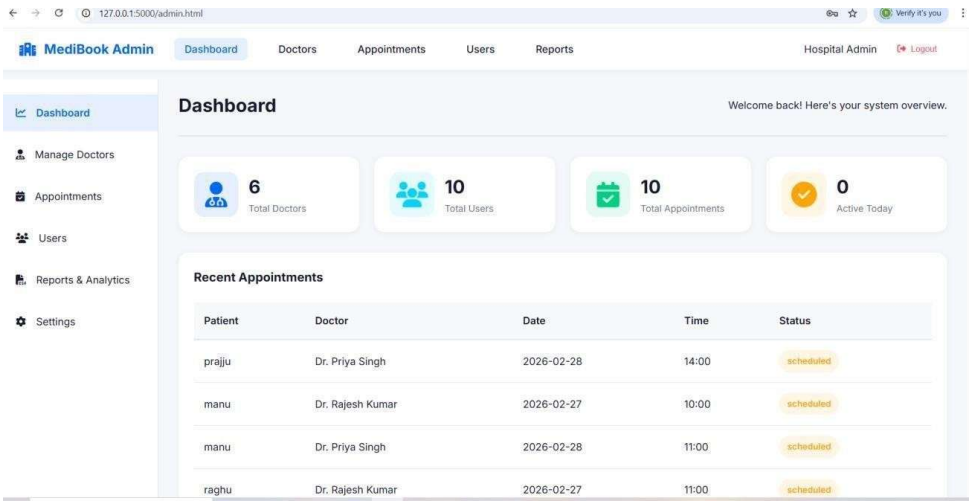
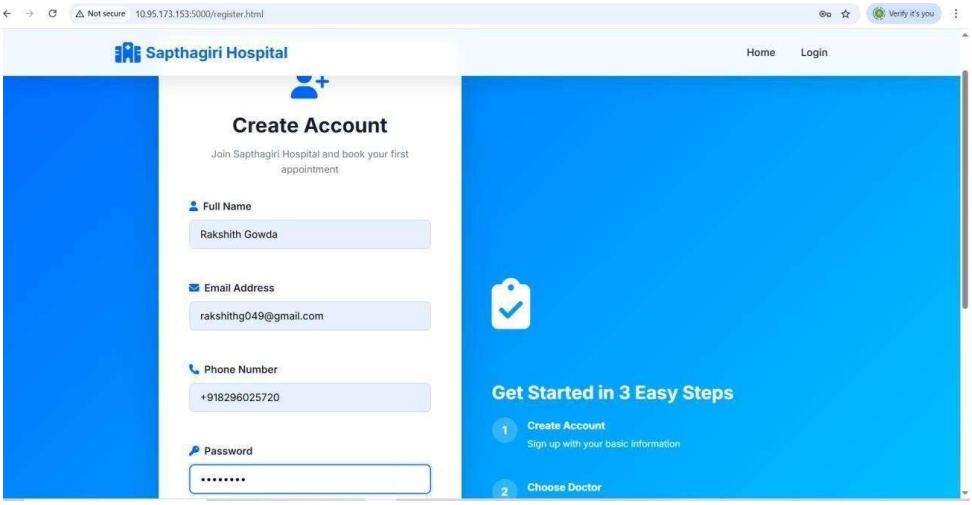

127.0.0.1:5000/admin.html
Verify it's you
MediBook Admin
Dashboard
Doctors
Appointments
Users
Reports
Hospital Admin
Logout
Dashboard
Manage Doctors
Appointments
Users
Reports & Analytics
Settings
Dashboard
Welcome back! Here's your system overview.
6
Total Doctors
10
Total Users
10
Total Appointments
0
Active Today
Recent Appointments
Patient
Doctor
Date
Time
Status
prajju
Dr. Priya Singh
2026-02-28
14:00
scheduled
manu
Dr. Rajesh Kumar
2026-02-27
10:00
scheduled
manu
Dr. Priya Singh
2026-02-28
11:00
scheduled
raghu
Dr. Rajesh Kumar
2026-02-27
11:00
scheduled

FIG5


Not secure
10.95.173.153:5000/register.html
Verify it's you
Sapthagiri Hospital
Home
Login
Create Account
Join Sapthagiri Hospital and book your first appointment
Full Name
Rakshith Gowda
Email Address
rakshithg049@gmail.com
Phone Number
+918296025720
Password
Get Started in 3 Easy Steps
1
Create Account
Sign up with your basic information
2
Choose Doctor

5 Conclusion

The Hospital Appointment Booking System was successfully designed and implemented using Python and MongoDB. The system provides an efficient way for patients to schedule appointments online without visiting the hospital physically. The application includes two main modules: user and admin. The user module allows patients to register, login, and book appointments with doctors. The admin module allows administrators to view, edit, and manage all appointment records.The use of MongoDB ensures flexible data storage and easy data retrieval. Python provides powerful tools for building web applications quickly and efficiently.

The proposed system improves hospital management by reducing manual work, minimizing errors, and saving time for both patients and hospital staff.In the future, the system can be enhanced by adding features such as doctor availability tracking, online payment integration, and SMS or email notifications for appointment reminders.Overall, the Hospital Appointment Booking System provides a reliable and user-friendly solution for managing hospital appointments.

6 Bibliography

1. Gupta, R., Sharma, P., & Verma, S. (2020). A Comparative Study of NoSQL Databases. International Journal of Computer Applications.

2. Rahman, M., & Ahmed, K. (2021). Document-Oriented Storage for Web Applications. Journal of Web Engineering and Data Systems.

3. Kumar, A., Singh, R., & Patel, D. (2019). Efficient Data Management Using MongoDB GridFS. International Journal of Database Management Systems.

4. Zhang, L., Chen, Y., & Li, H. (2022). Scalable Web-Based Transaction Systems. Journal of Distributed Computing and Applications.

5. Singh, T., & Kapoor, N. (2023). NoSQL Databases in Commercial Applications: A Survey. International Journal of Advanced Research in Computer Science.

6. MongoDB Inc. (2023). MongoDB Official Documentation. Available at: https://www.mongodb.com/docs/

7. Chodorow, K. (2013). MongoDB: The Definitive Guide. O'Reilly Media.

8. Banker, K. (2016). MongoDB in Action. Manning Publications.

9. Flask Documentation. (2023). Flask Web Framework. Available at: https://flask.palletsprojects.com/

10. Mozilla Developer Network (MDN). (2023). HTML, CSS, and JavaScript Documentation. Available at: https://developer.mozilla.org/.